Reviewer Pack — Minimal p-Adic Hodge Theory Rank and Circuit Monotone Width …

Entry 1d39ff147d05 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 02:47:02 UTC

Minimal p-Adic Hodge Theory Rank and Circuit Monotone Width Correlation

Entry ID: `1d39ff147d05` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 02:47:02 UTC

1. Conjecture

Field A (mathematical branch): p-adic Hodge Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For every d-regular Boolean circuit C with n inputs, the minimal rank of its associated p-adic Hodge theory space is linearly correlated with its monotone width w(C), such that min_rank(H_C) = Θ(w(C)).

Rationale (proposer's reasoning):

> p-adic Hodge theory provides a framework to study arithmetic properties of algebraic varieties over finite fields, which can be related to the complexity of Boolean functions through their associated algebraic structures. The conjecture suggests that the arithmetic complexity encoded in the p-adic Hodge theory space is reflected in the circuit's monotone width, offering potential insights into circuit complexity lower bounds.

Taxonomy category: `p-adic_hodge_theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2bf598d19c0ffe1a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between min_rank(H_C) and w(C) across multiple random seeds and circuits is greater than 0.5 with p-value < 0.01, indicating a strong positive linear correlation.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `p-adic Hodge Theory AND Boolean circuit monotone width` - `min_rank H_C correlation monotone width d-regular circuits` - `p-adic Hodge theory space rank linearly correlated with monotone width Boolean circuits`

Top relevant hits considered: - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P-Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/1908.08424v1] An introduction to p-adic period rings - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/nlin/0208039v1] Effects of RF Stimulus and Negative Feedback on Nonlinear Circuits - [http://arxiv.org/abs/2411.06569v2] Randomized Black-Box PIT for Small Depth +-Regular Non-commutative Circuits - [http://arxiv.org/abs/2405.20178v Non-intrusive data-driven model order reduction for circuits based on Hammerstein architectures

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_circuit(n, d):
        if n < 1 or d < 1 or d >= n:
            return None
        circuit = [[0] * n for _ in range(n)]
        edges = set()
        while len(edges) < d * n // 2:
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
            if u != v and (u, v) not in edges and (v, u) not in edges:
                circuit[u][v] = 1
                circuit[v][u] = 1
                edges.add((u, v))
        return circuit

    def compute_p_adic_hodge_rank(circuit):
        n = len(circuit)
        A = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
```

```
            if circuit[i][j] == 1:
                A[i][j] = 1
    rank = 0
    for row in A:
        if any(row):
            rank += 1
            for j in range(n):
                if row[j]:
                    for k in range(n):
                        A[k][j] -= row[k]
    return rank

def compute_monotone_width(circuit):
    n = len(circuit)
    width = 0
    for i in range(1 << n):
        subset = [j for j in range(n) if (i & (1 << j)) != 0]
        if all(circuit[u][v] == 0 for u in subset for v in subset if u != v):
            width = max(width, len(subset))
    return width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_d_regular_circuit(n, d=3)
    if circuit is None:
        continue
    min_rank = compute_p_adic_hodge_rank(circuit)
    w_C = compute_monotone_width(circuit)
    results.append((min_rank, w_C))

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

min_ranks = [r[0] for r in results]
w_Cs = [r[1] for r in results]
n_max = max(n_values)
instances_tested = len(results)

# Calculate Pearson correlation coefficient
mean_min_rank = sum(min_ranks) / instances_tested
mean_w_C = sum(w_Cs) / instances_tested
numerator = sum((min_ranks[i] - mean_min_rank) * (w_Cs[i] - mean_w_C) for i in range(instances
denominator = math.sqrt(sum((min_ranks[i] - mean_min_rank) ** 2 for i in range(instances_teste
              math.sqrt(sum((w_Cs[i] - mean_w_C) ** 2 for i in range(instances_tested)))

if denominator == 0:
    correlation_coefficient = None
```

```
    else:
        correlation_coefficient = numerator / denominator

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient is not None and correlation_coefficient > 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"m
        results.append(trial_result)

    if all(r["metric_value"] is not None for r in results):
        mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
        std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in res
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_frac
        else:
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={seeds[results.index(
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete its execution to calculate the Pearson correlation coefficient and p-value. | next: Re-run the test with

increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14061
2	propose	ollama_remote	glm4:latest	0	0	17617
3	preregistration	ollama_remote	glm4:latest	0	0	9533
4	novelty	ollama_remote	glm4:latest	0	0	8285
5	novelty	ollama_remote	glm4:latest	0	0	15171
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20971
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11791
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18325
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14373
10	judge	ollama_remote	glm4:latest	0	0	11882

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 142008 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1d39ff147d05.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1d39ff147d05.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1d39ff147d05.tar.gz` (if generated)